SIMATS ENGINEERING
ASSESSMENT TOOL 1

Course Name	INTERNET PROGRAMMING
Course Code	CSA43
Name	B KIRTHANAA
Register Number	192324244
Course Outcome Covered	CO-2 : Apply CSS layout and styling techniques to create visually appealing and responsive web interfaces, and use JavaScript to enable interactive client-side behavior. (L3)
Assessment Tool	Integrated Experiments
Weightage	40%

Experiment 1: Responsive Layout Design

Suppose that a retail website is not responsive across mobile and tablet devices, causing misalignment of content and improper spacing.

(a) Analyze the possible reasons for lack of responsiveness in the existing layout. (b) Design a responsive webpage layout using CSS Flexbox or Grid, ensuring proper alignment of header, navigation, content, and footer. (c) Demonstrate how the layout adapts to different screen sizes (mobile, tablet, desktop) using media queries. (d) Implement spacing, alignment, and distribution techniques to improve visual consistency. (e) Evaluate the effectiveness of the responsive design and suggest further improvements.

SOLUTION:

(a) Analyze the possible reasons for lack of responsiveness in the existing layout

A layout is responsive when it changes its structure to suit the width of the device. A page that breaks on mobile and tablet screens usually has one or more of the following problems:

- **Missing viewport meta tag:** without <meta name="viewport" content="width=device-width, initial-scale=1.0"> a phone renders the page at a desktop width (about 980px) and scales it down, so text becomes tiny and content looks misaligned.
- **Fixed pixel widths:** widths such as width: 960px force horizontal scrolling or overflow on screens narrower than the element.
- **No media queries:** the same multi-column layout is applied on every device, so columns become too narrow to read.
- **Old layout techniques:** floats, tables for layout and absolute positioning do not reflow when the available space changes.
- **Inconsistent spacing:** margins and padding in px, with no box-sizing: border-box, make elements grow beyond their intended width.
- **Non-flexible images:** images without max-width: 100% overflow their containers on small screens.

(b) Design a responsive webpage layout using CSS Flexbox or Grid

The page is built with a mobile-first approach. CSS Grid (grid-template-areas) places the header, navigation, main content and footer in a single column, so each part keeps its position. Flexbox is used inside the navigation bar, and a second Grid arranges the product cards. HTML and CSS code (retail.html):

HTML & CSS CODE:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>ShopEasy - Responsive Layout</title>
  <style>
    * { box-sizing: border-box; margin: 0; padding: 0; }
    body {
      font-family: Arial, sans-serif;
      line-height: 1.5;
      color: #222;
      display: grid;
      grid-template-areas: "header" "nav" "main" "footer";
      grid-template-rows: auto auto 1fr auto;
      min-height: 100vh;
    }
    header { grid-area: header; background: #1f3a5f; color: #fff;
             padding: 1rem; text-align: center; }
    nav { grid-area: nav; background: #2f5d8c; }
    nav ul { list-style: none; display: flex; flex-direction: column; }
    nav a { display: block; padding: 0.75rem 1rem; color: #fff;
            text-decoration: none; text-align: center; }
    nav a:hover { background: #1f3a5f; }
    main { grid-area: main; padding: 1rem; }
    main h2 { margin-bottom: 1rem; }
    .products { display: grid; grid-template-columns: 1fr; gap: 1rem; }
    .card { border: 1px solid #ccc; border-radius: 8px; padding: 1rem;
            text-align: center; background: #f9f9f9; }
    .card .img { height: 90px; background: #cfd8e3; border-radius: 6px;
                 margin-bottom: 0.75rem; }
    .card button { margin-top: 0.5rem; padding: 0.5rem 1rem; border: 0;
                   background: #1f3a5f; color: #fff; border-radius: 4px; }
    footer { grid-area: footer; background: #1f3a5f; color: #fff;
             text-align: center; padding: 1rem; }

    /* Tablet: 768px and above */
    @media (min-width: 768px) {
      nav ul { flex-direction: row; justify-content: space-around; }
      nav li { flex: 1; }
      .products { grid-template-columns: repeat(2, 1fr); }
    }
    /* Desktop: 1024px and above */
    @media (min-width: 1024px) {
      main { max-width: 1100px; margin: 0 auto; padding: 2rem; width: 100%; }
      .products { grid-template-columns: repeat(4, 1fr); gap: 1.5rem; }
    }
  </style>
</head>
<body>
  <header><h1>ShopEasy</h1></header>
  <nav>
    <ul>
      <li><a href="#">Home</a></li>
      <li><a href="#">Products</a></li>
      <li><a href="#">Offers</a></li>
      <li><a href="#">Contact</a></li>
    </ul>
  </nav>
  <main>
    <h2>Featured Products</h2>
    <section class="products">
      <article class="card"><div class="img"></div><h3>Headphones</h3><p>Rs.
1,999</p><button>Add to Cart</button></article>
      <article class="card"><div class="img"></div><h3>Smart Watch</h3><p>Rs.
3,499</p><button>Add to Cart</button></article>
      <article class="card"><div class="img"></div><h3>Backpack</h3><p>Rs. 899</p><button>Add to
Cart</button></article>
      <article class="card"><div class="img"></div><h3>Sneakers</h3><p>Rs. 2,299</p><button>Add
to Cart</button></article>
    </section>
  </main>
  <footer><p>&copy; 2026 ShopEasy. All rights reserved.</p></footer>
</body>
</html>
```

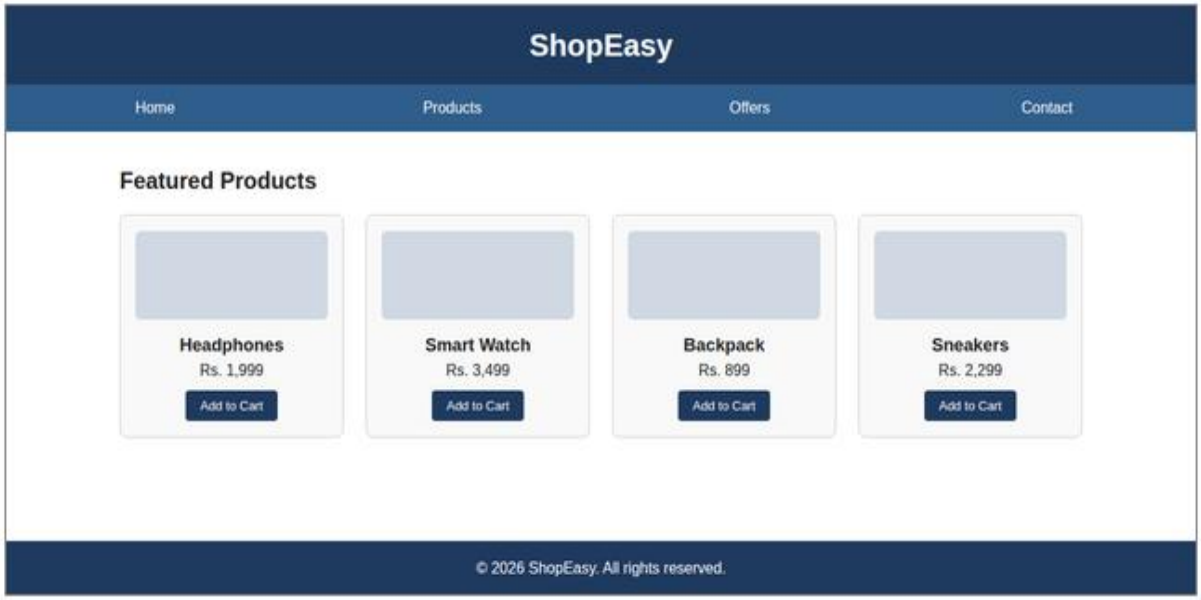
(c) Demonstrate how the layout adapts to different screen sizes using media queries

The base styles target mobile phones. Two media queries then add rules as the screen widens:

- **Mobile (below 768px):** navigation links are stacked vertically and the product cards are shown in 1 column.

- **Tablet (768px and above):** the navigation becomes a horizontal Flexbox row and the cards are shown in 2 columns.
- **Desktop (1024px and above):** the content is centred with a maximum width of 1100px and the cards are shown in 4 columns.

OUTPUT:



(d) Implement spacing, alignment, and distribution techniques

The following techniques in the code keep the design visually consistent on every device:

- **Universal reset:** * { box-sizing: border-box; margin: 0; padding: 0; } removes default browser differences and makes widths include padding and borders.
- **Relative units:** rem units for padding and gaps scale with the user's font size instead of staying fixed.
- **Grid gap:** gap gives equal spacing between cards without extra margins on individual items.
- **Flexbox distribution:** justify-content: space-around and flex: 1 distribute navigation links evenly across the bar.
- **Alignment:** text-align: center for cards and margin: 0 auto for the desktop container keep the content balanced.
- **Consistent colour scheme:** the header, navigation and footer share one colour family, and the cards use equal padding and border radius.

(e) Evaluate the effectiveness of the responsive design and suggest further improvements

Evaluation: the layout works correctly at all three tested widths. There is no horizontal scrolling, content is readable without zooming, and the header, navigation, content and footer remain properly aligned. Using Grid and Flexbox keeps the CSS short and removes the need for floats or fixed widths.

Suggested improvements:

- Add a hamburger menu for the navigation on mobile screens to save vertical space.
- Use responsive images (srcset, max-width: 100%) and lazy loading to improve load time on mobile networks.
- Use grid-template-columns: repeat(auto-fit, minmax(220px, 1fr)) so the number of columns adjusts automatically without fixed breakpoints.
- Use clamp() for fluid font sizes and add a prefers-color-scheme dark mode.
- Test on real devices and with Chrome DevTools device mode, and check accessibility (contrast, focus states, touch targets of at least 44px).

Experiment 2: Form Validation using JavaScript

Suppose that a registration form accepts invalid email and phone number inputs, leading to incorrect data submission.

(a) Analyze the issues in the existing form validation mechanism. (b) Design input fields for email and phone number with appropriate HTML attributes. (c) Implement JavaScript validation using Regular Expressions for both email and phone number. (d) Develop a mechanism to display real-time error messages for invalid inputs. (e) Evaluate the validation system and suggest enhancements for better user experience and security.

SOLUTION:

(a) Analyze the issues in the existing form validation mechanism

- **Wrong or missing input types:** plain type="text" fields accept any characters, so the browser performs no built-in email or phone checks.
- **No required attribute:** empty fields can be submitted.
- **No pattern or length checks:** phone numbers containing letters, symbols or too few digits are accepted.
- **No JavaScript validation:** values are not tested against a format before submission.
- **No feedback:** the user is not told what is wrong or when, so incorrect data reaches the server.
- **Client-side only:** even with validation, the server must re-check the data because browser checks can be bypassed.

(b) Design input fields for email and phone number with appropriate HTML attributes

The email field uses type="email" and the phone field uses type="tel". Other attributes make the form more usable and accessible:

- **required:** the field cannot be left empty.
- **placeholder:** shows an example of the expected format.
- **maxlength="10":** limits the phone field to 10 characters.
- **inputmode="numeric":** opens the numeric keypad on mobile devices.
- **autocomplete:** lets the browser fill in saved values.
- **label for / id and aria-live:** connect each label to its input and make screen readers announce error messages.

(c) Implement JavaScript validation using Regular Expressions

Two regular expressions are used:

- **Email:** /^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$/ checks for a valid local part, an @ symbol, a domain name and a top-level domain of at least 2 letters.
- **Phone:** /^[6-9][0-9]{9}$/ accepts exactly 10 digits, with the first digit between 6 and 9 (Indian mobile number format).

(d) Develop a mechanism to display real-time error messages

An "input" event listener runs the validation function every time the user types a character. If the value is empty or does not match its regular expression, a red message appears below the field and the border turns red. When the value is correct the message is removed and the border turns green. The Register button stays disabled until both fields are valid. The complete code (validation.html) is given below.

HTML, CSS & JAVASCRIPT CODE:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Registration Form Validation</title>
  <style>
    body { font-family: Arial, sans-serif; background: #eef2f7; }
    form { width: 340px; margin: 30px auto; padding: 20px; background: #fff;
```

```
          border-radius: 8px; box-shadow: 0 2px 8px rgba(0,0,0,.15); }
    h2 { text-align: center; margin-bottom: 15px; }
    label { display: block; margin-top: 12px; font-weight: bold; }
    input { width: 100%; padding: 8px; margin-top: 5px; box-sizing: border-box;
            border: 2px solid #aaa; border-radius: 4px; }
    input.valid { border-color: #2e8b57; }
    input.invalid { border-color: #c0392b; }
    .error { color: #c0392b; font-size: 13px; min-height: 18px; margin-top: 4px; }
    button { width: 100%; margin-top: 15px; padding: 10px; border: 0;
             background: #1f3a5f; color: #fff; border-radius: 4px; cursor: pointer; }
    button:disabled { background: #999; cursor: not-allowed; }
  </style>
</head>
<body>
  <form id="regForm" novalidate>
    <h2>Registration</h2>

    <label for="email">Email</label>
    <input type="email" id="email" name="email" placeholder="name@example.com"
           required autocomplete="email">
    <div class="error" id="emailError" aria-live="polite"></div>

    <label for="phone">Phone Number</label>
    <input type="tel" id="phone" name="phone" placeholder="9876543210"
           required maxlength="10" inputmode="numeric" autocomplete="tel">
    <div class="error" id="phoneError" aria-live="polite"></div>

    <button type="submit" id="submitBtn" disabled>Register</button>
  </form>

  <script>
    const emailRegex = /^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$/;
    const phoneRegex = /^[6-9][0-9]{9}$/;   // 10-digit Indian mobile number

    const form = document.getElementById("regForm");
    const email = document.getElementById("email");
    const phone = document.getElementById("phone");
    const btn = document.getElementById("submitBtn");

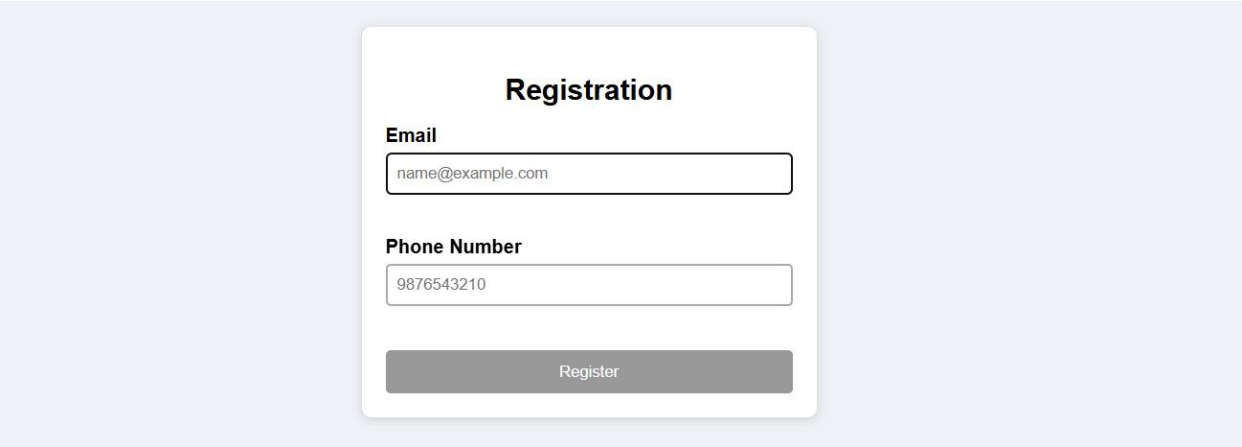
    function validate(input, regex, errorId, message) {
      const error = document.getElementById(errorId);
      const value = input.value.trim();
      if (value === "") {
        error.textContent = "This field is required.";
        input.className = "invalid";
        return false;
      }
      if (!regex.test(value)) {
        error.textContent = message;
        input.className = "invalid";
        return false;
      }
      error.textContent = "";
      input.className = "valid";
      return true;
    }

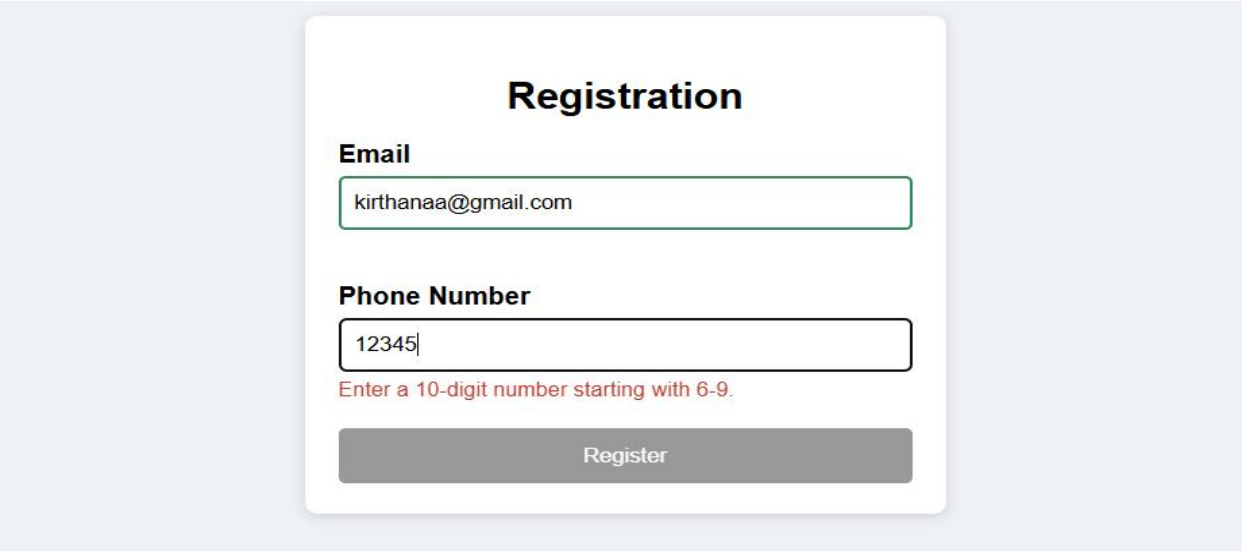
    function checkForm() {
      const okEmail = validate(email, emailRegex, "emailError",
                               "Enter a valid email, e.g. name@example.com");
      const okPhone = validate(phone, phoneRegex, "phoneError",
                               "Enter a 10-digit number starting with 6-9.");
      btn.disabled = !(okEmail && okPhone);
    }

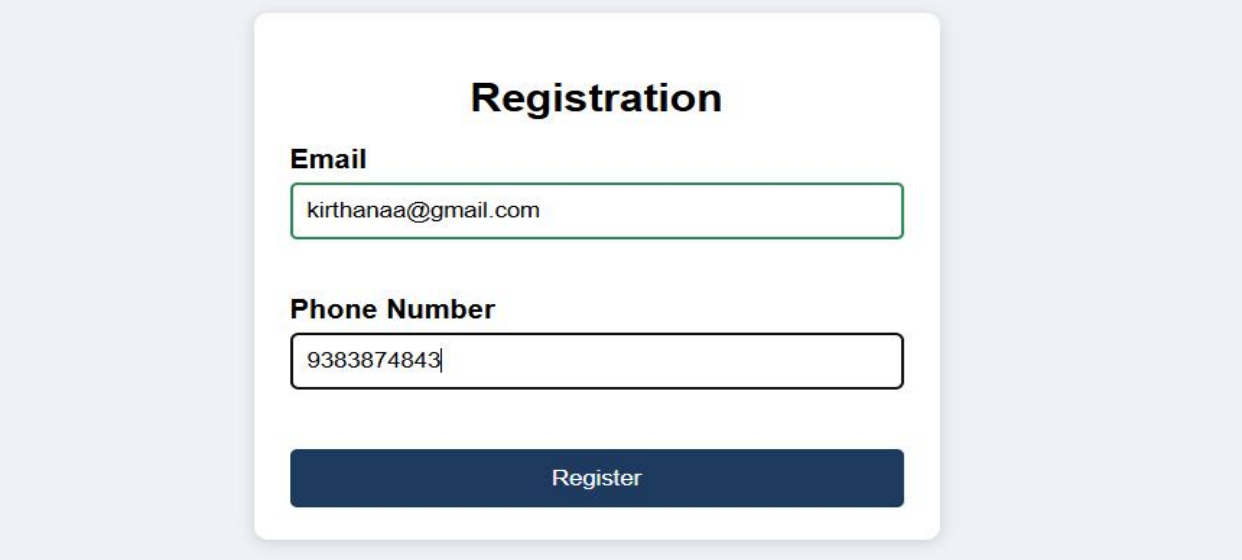
    // Real-time validation on every keystroke (only after the user types)
    [email, phone].forEach(function (field) {
      field.addEventListener("input", checkForm);
    });

    form.addEventListener("submit", function (e) {
      e.preventDefault();
      checkForm();
      if (!btn.disabled) { alert("Registration successful!"); }
    });
  </script>
</body>
</html>
```

OUTPUT:







(e) Evaluate the validation system and suggest enhancements

Evaluation: the form now rejects empty fields, malformed email addresses (such as kirthanaa@gmail) and phone numbers that are too short or begin with an invalid digit. Because feedback is shown as the user types, mistakes are corrected immediately and invalid data does not reach the server. The combination of HTML5 attributes and regular expressions keeps the code simple and readable.

Suggested enhancements:

- **Server-side validation:** repeat all checks on the server, since JavaScript can be disabled or bypassed.

- **Sanitise input:** escape or sanitise values to prevent XSS and SQL injection, and use parameterised queries.
- **Email verification:** send a confirmation link or OTP to confirm that the email address and phone number really exist.
- **Better user experience:** show errors only after the user leaves the field (blur event), allow spaces or +91 in phone numbers, and give helpful hints for each rule.
- **Security:** use HTTPS, CSRF tokens and rate limiting or CAPTCHA to stop automated submissions.
- **Accessibility:** do not rely on colour alone to show errors; keep aria-live messages and clear focus management.

Experiment 3: CSS Layout Debugging

Suppose that a webpage layout overlaps when the browser window is resized, affecting usability.

(a) Analyze the CSS properties responsible for layout overlapping issues. (b) Examine the role of the CSS box model (margin, border, padding, content) in layout design. (c) Debug the layout using appropriate positioning techniques (relative, absolute, flex, grid). (d) Modify the CSS to ensure proper alignment and prevent overlapping during resizing. (e) Evaluate the corrected layout and suggest best practices for maintaining consistency.

SOLUTION:

(a) Analyze the CSS properties responsible for layout overlapping issues

The faulty code (shown below) places every block with position: absolute. Absolutely positioned elements are removed from the normal document flow, so they do not push or wrap around each other. The main causes of overlap are:

- **position: absolute with left/top values:** the sidebar (250px wide) and the content (starting at left: 200px) already overlap by 50px plus borders and padding.
- **Fixed width and height:** width: 600px and height: 150px do not shrink when the window is resized, so the blocks extend beyond the viewport.
- **Fixed top offset on the footer:** top: 140px places the footer on top of the 150px high blocks.
- **Missing box-sizing: border-box:** padding and borders are added on top of the declared width and height, making each box larger than expected.
- **No wrapping or media queries:** nothing tells the blocks to reflow when space is limited.

Faulty CSS and HTML (exp3_before.html):

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Layout Overlap (Before)</title>
  <style>
    .container { position: relative; height: 200px; }
    .sidebar { position: absolute; left: 0; top: 0; width: 250px; height: 150px;
               background: #f4b942; padding: 20px; border: 5px solid #333; }
    .content { position: absolute; left: 200px; top: 0; width: 600px; height: 150px;
               background: #6fb1e4; padding: 20px; border: 5px solid #333; }
    .footer  { position: absolute; top: 140px; width: 100%; height: 50px;
               background: #8fd19e; }
  </style>
</head>
<body>
  <div class="container">
    <div class="sidebar">Sidebar</div>
    <div class="content">Main Content</div>
    <div class="footer">Footer</div>
  </div>
</body>
</html>
```

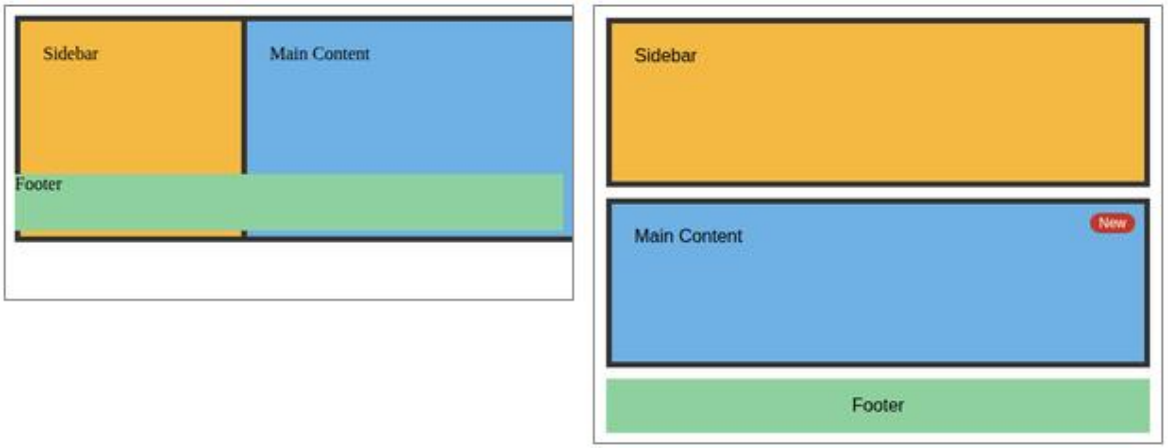
(b) Examine the role of the CSS box model in layout design

Every HTML element is a rectangular box made of four layers, from the inside to the outside:

- **Content:** the actual text or image, sized by width and height.
- **Padding:** transparent space between the content and the border.
- **Border:** the line that surrounds the padding and content.
- **Margin:** transparent space outside the border that separates the element from its neighbours.

With the default content-box model, the total width of an element = width + left/right padding + left/right border + left/right margin. In the faulty code the sidebar is 250 + 40 (padding) + 10 (border) = 300px wide, and the content box is 600 + 40 + 10 = 650px wide. Together they need 950px, which is more than a tablet or phone can show, so the blocks overlap or overflow. Setting box-sizing: border-box makes width include padding and border, so sizes are predictable.

OUTPUT:



(c) Debug the layout using appropriate positioning techniques

- **static (default):** elements follow the normal flow and cannot overlap; this is the safest option for main layout blocks.
- **relative:** keeps the element in the flow but allows offsets, and creates a reference point for absolutely positioned children.
- **absolute:** removes the element from the flow; it should be used only for small items such as badges, icons or tooltips inside a position: relative parent.
- **flex:** arranges items in a row or column, shares the available space (flex: 1 1 200px) and wraps items with flex-wrap: wrap instead of overlapping.
- **grid:** places items in rows and columns and is best for two-dimensional page layouts.

Debugging steps: (1) inspect the elements with browser DevTools to see their box-model dimensions; (2) remove absolute positioning from the main blocks; (3) switch the container to Flexbox with wrapping; (4) replace fixed widths with flexible flex-basis values; (5) keep absolute positioning only for the small "New" badge inside a relative parent; (6) move the footer back into the normal flow.

(d) Modify the CSS to ensure proper alignment and prevent overlapping

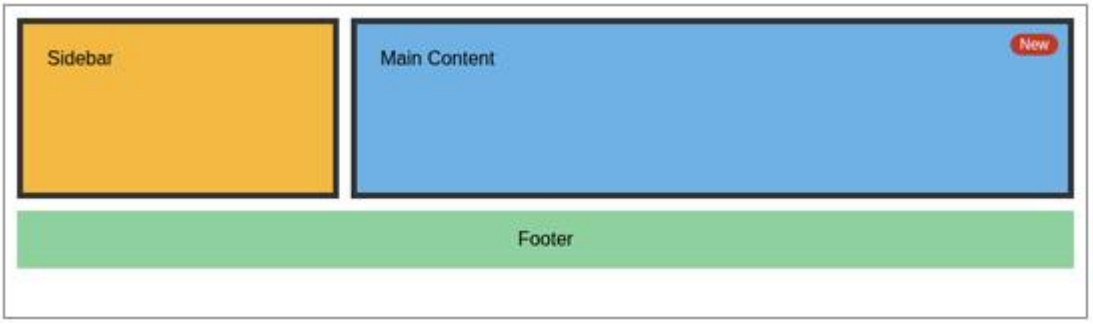
The corrected code is given below (exp3_after.html).

CORRECTED HTML & CSS CODE:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Layout Overlap (After)</title>
  <style>
    * { box-sizing: border-box; margin: 0; padding: 0; }
    body { font-family: Arial, sans-serif; padding: 10px; }
    .container { display: flex; flex-wrap: wrap; gap: 10px; }
    .sidebar { flex: 1 1 200px; background: #f4b942; padding: 20px;
```

```
              border: 5px solid #333; min-height: 150px; }
    .content { flex: 3 1 400px; background: #6fb1e4; padding: 20px;
              border: 5px solid #333; min-height: 150px; position: relative; }
    .badge   { position: absolute; top: 8px; right: 8px; background: #c0392b;
              color: #fff; padding: 2px 8px; border-radius: 10px; font-size: 12px; }
    .footer  { margin-top: 10px; background: #8fd19e; padding: 15px;
              text-align: center; }
  </style>
</head>
<body>
  <div class="container">
    <aside class="sidebar">Sidebar</aside>
    <main class="content"><span class="badge">New</span>Main Content</main>
  </div>
  <footer class="footer">Footer</footer>
</body>
</html>
```

OUTPUT:



In the corrected output the sidebar and content sit side by side in a wide window and wrap into a stacked layout in a narrow window. The footer always stays below them, and no element overlaps at any width.

(e) Evaluate the corrected layout and suggest best practices

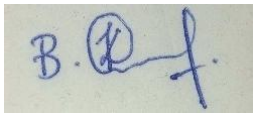
Evaluation: the corrected layout stays free of overlaps when the browser window is resized from desktop width to mobile width. The blocks share the space proportionally (flex: 1 and flex: 3), the gap creates equal spacing, and the footer stays in the normal flow. The only absolutely positioned element is the badge, which is anchored to its relative parent and cannot affect other elements.

Best practices for maintaining consistency:

- Apply * { box-sizing: border-box; } and a CSS reset or normalize file in every project.
- Use Flexbox for one-dimensional layouts and Grid for two-dimensional layouts; avoid absolute positioning for page structure.
- Prefer relative units (%, rem, fr, vw) and max-width over fixed pixel widths.
- Use media queries at sensible breakpoints and a mobile-first approach.
- Use CSS variables for colours and spacing, and follow a naming convention such as BEM.
- Test in several browsers and screen sizes with DevTools, and validate the CSS with the W3C CSS Validator.

Code of Conduct:

I **B KIRTHANAA 192324244** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.



Signature of the student: **B KIRTHANAA**

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30%	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20%	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15%	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10%	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation